

ASSIGNMENT-9

PART-A (Conceptual Questions)

Q1. What is the static keyword in Java?

Explain the difference between a and an.

Answer-

The static keyword in Java is used to define members that belong to the class rather than to individual objects. A static member is created only once and can be shared by all objects of that class.

For example, if every student belongs to the same college, the college name can be declared as static because it is common to all students.

Difference Between Static and Non-Static Members:

- Static members belong to the class and can be accessed using the class name.
- Non-static members belong to individual objects and generally require an object for access.
- Static members are useful when the same data or functionality is common to all objects.

Q2. What is a Static Variable?

Explain with an example why a static variable is shared among all objects of a class.

Answer-

A static variable is a variable declared using the static keyword. It is shared among all objects of a class, meaning only one copy of the variable is created in memory.

Example:

```
class Student {  
    int rollNo;  
    static String college = "IILM University";  
    Student(int r) {  
        rollNo = r;  
    }  
    void display() {  
        System.out.println(rollNo + " " + college);  
    }  
    public static void main(String[] args) {  
        Student s1 = new Student(101);  
        Student s2 = new Student(102);  
        s1.display();  
        s2.display();  
    }  
}
```

Explanation: Here, rollNo is different for every student because it is an instance variable. However, college is static, so both students share the same college name. If the value of college changes, the change will be reflected for all objects. This is why static variables are also called class variables.

Q3. What is a Static Method?

Explain how a static method is different from an instance method.

Answer-

A static method is a method declared using the static keyword. It belongs to the class and can be called without creating an object of that class.

A static method is associated with the class and can be called directly using the class name. It is mainly used when the method does not depend on the data of a particular object.

An instance method belongs to an object and is called using an object reference. It can access both instance and static members.

Example:

```
class Demo {  
    static void display() {  
        System.out.println("This is a static method.");  
    }  
    public static void main(String[] args) {  
        Demo.display();  
    }  
}
```

In simple words, static methods are useful for common operations, while instance methods are used for object-specific operations

Q4. Why is the main() method static?

Explain why Java can execute main() without creating an object of the class.

Answer-

The main() method is declared as static because the JVM needs to call it without creating an object of the class.

When a Java program starts, the JVM searches for the following method:

```
public static void main(String[] args)
```

Since the method is static, the JVM can execute it directly using the class name. There is no need to create an object before the program begins.

Therefore, the static keyword allows main() to act as the entry point of a Java program.

Q5. Can a static method directly access an instance variable?

Explain why or why not, with an example.

Answer-

No, a static method cannot directly access an instance variable.

This is because a static method belongs to the class, whereas an instance variable belongs to a specific object. A static method does not automatically know which object's instance variable it should access.

However, a static method can access an instance variable by creating an object and using that object to refer to the variable.

In conclusion, a static method cannot access instance variables directly, but it can access them indirectly through an object.

Example of an incorrect direct access:

```
class Demo {  
    int number = 10;  
    static void display() {  
        // System.out.println(number); // Compilation error  
    }  
}
```

Correct approach:

```
class Demo {  
    int number = 10;  
  
    static void display() {  
        Demo obj = new Demo();  
        System.out.println(obj.number);  
    }  
  
    public static void main(String[] args) {  
        display();  
    }  
}
```

Real-Life Analogy:

Think of a college name shared by all students. The college name is common to everyone, just like a static variable. But each student's roll number is personal, just like an instance variable.

Conclusion:

The static keyword is useful when data or methods need to be shared at the class level. It helps save memory, allows direct access through the class name, and is especially important because it makes the `main()` method executable without creating an object.

PART-B (Programming Questions)

Q) Bank Account Interest Rate:

Create a class BankAccount with: accountNumber, accountHolderName, balance and interestRate

Answer:

```
class BankAccount {  
    // Instance variables  
    int accountNumber;  
    String accountHolderName;  
    double balance;  
    // Static variable  
    static double interestRate = 5.0;  
    // Constructor  
    BankAccount(int accountNumber, String accountHolderName, double balance) {  
        this.accountNumber = accountNumber;  
        this.accountHolderName = accountHolderName;  
        this.balance = balance;  
    }  
    // Method to display account details  
    void displayDetails() {  
        System.out.println("Account Number: " + accountNumber);  
        System.out.println("Account Holder: " + accountHolderName);  
        System.out.println("Balance: " + balance);  
        System.out.println("Interest Rate: " + interestRate + "%");  
        System.out.println("-----");  
    }  
}
```

```
public class bank{  
    public static void main(String[] args) {  
        // Creating three bank accounts  
        BankAccount a1 = new BankAccount(101, "Harsh", 50000);  
        BankAccount a2 = new BankAccount(102, "Anubhav", 75000);  
        BankAccount a3 = new BankAccount(103, "Deep", 60000);  
        System.out.println("Before Changing Interest Rate:");  
        a1.displayDetails();  
        a2.displayDetails();  
        a3.displayDetails();  
        // Changing static interest rate using class name  
        BankAccount.interestRate = 6.5;  
        System.out.println("\nAfter Changing Interest Rate:");  
        a1.displayDetails();  
        a2.displayDetails();  
        a3.displayDetails();  
    }  
}
```

Output:

```
Problems 2 Output Debug Console Terminal Ports

PS C:\Users\harsh\OneDrive\Desktop\Java\Assignment_9> java bank.java
Before Changing Interest Rate:
Account Number: 101
Account Holder: Harsh
Balance: 50000.0
Interest Rate: 5.0%
-----
Account Number: 102
Account Holder: Anubhav
Balance: 75000.0
Interest Rate: 5.0%
-----
Account Number: 103
Account Holder: Deep
Balance: 60000.0
Interest Rate: 5.0%
-----

After Changing Interest Rate:
Account Number: 101
Account Holder: Harsh
Balance: 50000.0
Interest Rate: 6.5%
-----
Account Number: 102
Account Holder: Anubhav
Balance: 75000.0
Interest Rate: 6.5%
-----
Account Number: 103
Account Holder: Deep
Balance: 60000.0
Interest Rate: 6.5%
-----

PS C:\Users\harsh\OneDrive\Desktop\Java\Assignment_9> |
```